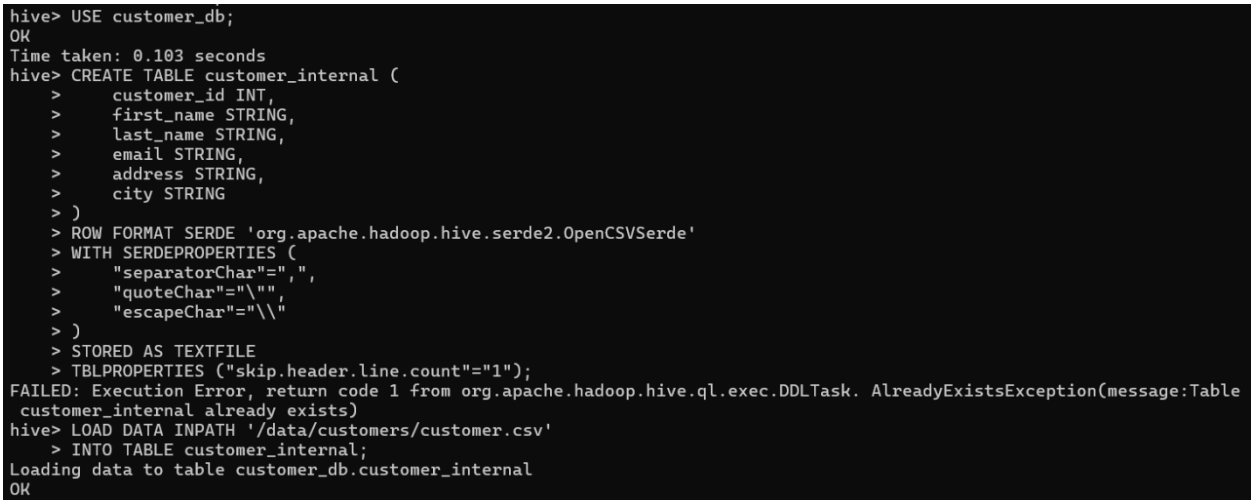
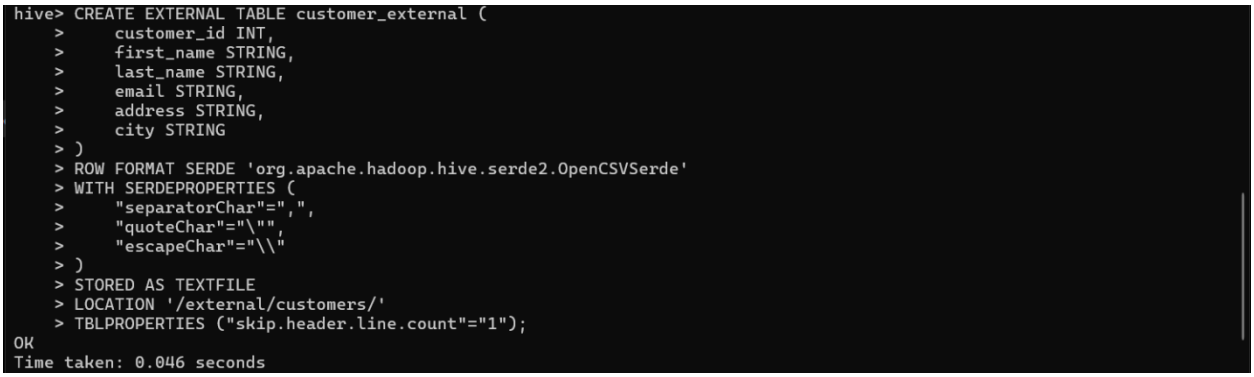
- **Create internal and external customer tables and load data into them.**

  To store customer data in Hive using both managed (internal) and unmanaged (external) storage models.



```
hive> USE customer_db;
OK
Time taken: 0.103 seconds
hive> CREATE TABLE customer_internal (
    >     customer_id INT,
    >     first_name STRING,
    >     last_name STRING,
    >     email STRING,
    >     address STRING,
    >     city STRING
    > )
    > ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
    > WITH SERDEPROPERTIES (
    >     "separatorChar"=",",
    >     "quoteChar"="\"",
    >     "escapeChar"="\\"
    > )
    > STORED AS TEXTFILE
    > TBLPROPERTIES ("skip.header.line.count"="1");
FAILED: Execution Error, return code 1 from org.apache.hadoop.hive.ql.exec.DDLTask. AlreadyExistsException(message:Table
 customer_internal already exists)
hive> LOAD DATA INPATH '/data/customers/customer.csv'
    > INTO TABLE customer_internal;
Loading data to table customer_db.customer_internal
OK
```



```
hive> CREATE EXTERNAL TABLE customer_external (
    >     customer_id INT,
    >     first_name STRING,
    >     last_name STRING,
    >     email STRING,
    >     address STRING,
    >     city STRING
    > )
    > ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
    > WITH SERDEPROPERTIES (
    >     "separatorChar"=",",
    >     "quoteChar"="\"",
    >     "escapeChar"="\\"
    > )
    > STORED AS TEXTFILE
    > LOCATION '/external/customers/'
    > TBLPROPERTIES ("skip.header.line.count"="1");
OK
Time taken: 0.046 seconds
```

Justification:

- Internal Table: Hive fully manages both schema and physical data.
- External Table: Hive manages metadata only, while data remains independently stored in HDFS.
  Through this implementation, I observed the practical architectural difference between managed and unmanaged Hive storage models in Hive.

- **Notice the issue related to delimiter inside the address column value, find a solution to make hive read the file properly without changing the separator. (Do some web search)**

Problem:

- Customer address fields contained commas inside quoted values, such as:
- "123 Main St, New York"
- Default Hive parsing treats commas as field separators, causing incorrect column shifts.

Solution:

- Using OpenCSVSerde, which properly handles quoted CSV fields. Apache Hive documentation and community best practices recommend OpenCSVSerde for CSV files containing embedded delimiters

  During implementation, I found that OpenCSVSerde was the most reliable solution for preserving embedded commas inside quoted address values without modifying the original dataset.

```
hive> ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
    > WITH SERDEPROPERTIES (
    >     "separatorChar"=",",
    >     "quoteChar"="\"",
    >     "escapeChar"="\\"
    > )
```

Justification:

- Preserved original file structure
- No need to modify source delimiter
- Correctly parsed quoted addresses
- Improved data quality and integrity

- **Try to drop both external and internal tables and notice what happens to the data.**

```
hive> DROP TABLE customer_internal;
OK
Time taken: 0.061 seconds
hive> DROP TABLE customer_external;
OK
Time taken: 0.046 seconds
hive>
```

```
C:\Users\m7mdl\Desktop\hive-lab>docker exec -it namenode bash
root@namenode:/# hdfs dfs -ls /external/customers
Found 1 items
-rw-r--r--   3 root supergroup     513165 2026-05-08 12:41 /external/customers/customer.csv
root@namenode:/#
```

Result:

• Internal table deletion removed both metadata and physical data.

• External table deletion removed metadata only while preserving raw data in HDFS.

After testing both table types, I confirmed that dropping internal tables permanently removes both metadata and physical data, while external tables preserve raw files inside HDFS

Justification:

- This step proves:
- Internal tables are Hive-managed storage
- External tables are preferable for reusable raw data lakes

Create customer dimension (Slowly changing dimension type 2) as "customer_scd2_mixed.csv" file

```
Time taken: 0.040 seconds
hive> CREATE TABLE customer_dim (
    >     customer_id INT,
    >     first_name STRING,
    >     last_name STRING,
    >     email STRING,
    >     address STRING,
    >     city STRING,
    >     start_date DATE,
    >     end_date DATE,
    >     is_current STRING
    > )
    > ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
    > WITH SERDEPROPERTIES (
    >     "separatorChar"=",",
    >     "quoteChar"="\"",
    >     "escapeChar"="\\"
    > )
    > STORED AS TEXTFILE
    > TBLPROPERTIES ("skip.header.line.count"="1");
OK
Time taken: 0.045 seconds
hive> LOAD DATA INPATH '/data/customers/customer_scd2_mixed.csv'
    > INTO TABLE customer_dim;
Loading data to table customer_db.customer_dim
```

This dimension design allows customer records to maintain historical versions while accurately identifying the currently active record.

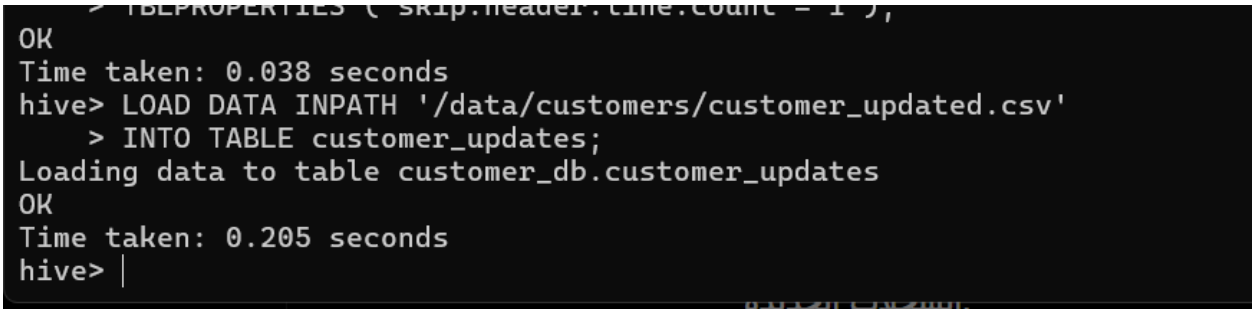
Justification:

- start_date: record validity start
- end_date: record expiration
- is_current: active/inactive indicator
- This structure enables full historical versioning.

- **Use "customer_updated.csv" file to insert new data with end_date = null and is_current = '1' and update changed data to be with end_date and is_current = "0" to track history.**

```
hive> CREATE TABLE customer_updates (
    >     customer_id INT,
    >     first_name STRING,
    >     last_name STRING,
    >     email STRING,
    >     address STRING,
    >     city STRING
    > )
    > ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
    > WITH SERDEPROPERTIES (
    >     "separatorChar"=",",
    >     "quoteChar"="\"",
    >     "escapeChar"="\\"
    > )
    > STORED AS TEXTFILE
    > TBLPROPERTIES ("skip.header.line.count"="1");
OK
Time taken: 0.038 seconds
hive>
```

By inserting new versions of modified customers while expiring previous records, the warehouse preserves historical accuracy without losing prior customer states.



Business Logic:

New rows:

- end_date = NULL
- is_current = 1

Changed rows:

- Previous record closed:
- end_date = current_date
- is_current = 0

Justification:

- Full history
- Current active customers
- Audit trail for customer changes

- **“Hive doesn’t support update or deletes so find a work around to make this SCD type 2 without using Transactional tables”**

Problem:

Hive non-transactional tables do not support direct UPDATE or DELETE.

To overcome this limitation, I implemented a CTAS (Create Table As Select) reconstruction strategy to rebuild the customer dimension table while preserving both active and historical records.

```
hive> CREATE TABLE customer_dim_new AS
    >
    > SELECT *
    > FROM customer_dim
    > WHERE is_current='0'
    >
    > UNION ALL
    >
    > SELECT
    >     old.customer_id,
    >     old.first_name,
    >     old.last_name,
    >     old.email,
    >     old.address,
    >     old.city,
    >     old.start_date,
    >     CAST(current_date AS STRING),
    >     '0'
    > FROM customer_dim old
    > JOIN customer_updates upd
    > ON old.customer_id = upd.customer_id
    > WHERE old.is_current='1'
    >
    > UNION ALL
    >
    > SELECT
    >     upd.customer_id,
    >     upd.first_name,
    >     upd.last_name,
    >     upd.email,
    >     upd.address,
    >     upd.city,
    >     CAST(current_date AS STRING),
    >     NULL,
    >     '1'
    > FROM customer_updates upd;
```

Justification:

- Avoids transactional dependencies
- Preserves SCD Type 2 logic
- Supports scalable warehouse design
- Simulates UPDATE/DELETE through table reconstruction